

Intermediate (wrap real logic behind a route) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `chat_route_practice.py`: this is the exact shape of the Build Task's `POST /run-task` — a typed request in, real logic in the middle, a typed reply out, and every failure turned into the right status code. Practicing it on a tiny fake chat function keeps the focus on the route, not the logic. **If not:** the Build Task would be the first place you ever mapped errors to 400 vs 500, with a multi-agent pipeline making every mistake harder to see.

`run_chat()` below is a small stand-in for Doc04's real chat function — swap in the real one, the route around it doesn't change. Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# practice/chat_route_practice.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

def run_chat(message):
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest):
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
```

```
except Exception:
    raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output (`curl -X POST localhost:8000/chat -d '{"message": "hi"}'`):

```
{"reply": "You said: hi"}
```

Expected output for a missing `message` field:

```
422 Unprocessable Entity
```

This meets the exercise's core requirement — typed request in, typed response out, errors don't crash the server. It treats every internal failure as a 500 with no distinction from a bad input, and it's missing type hints on the route function itself — both addressed below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — splitting 400 (bad input) from 500 (real failure)

Story: a client needs to know whether to fix its request or just try again later — one generic 500 for everything can't tell it that. A named `ChatInputError` marks "your input is the problem" on purpose. **If not:** an empty message and a real crash would look identical to the client, and it would keep resending a request that can never work.

```
# practice/chat_route_practice.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    """Raised on purpose when input is well-formed but not usable."""
    pass

def run_chat(message: str) -> str:
    # why: "" passes Pydantic's check (it IS a str), so this rule
    # has to live in our own code
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        # how: our own message is safe to show – we wrote it
        raise HTTPException(status_code=400, detail=str(exc))
```

```
except Exception:
    raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output for `{"message": ""}`:

```
400 {"detail": "message cannot be empty"}
```

Expected output for `{"message": "hi"}`:

```
200 {"reply": "You said: hi"}
```

Approach 2 — moving the models into their own file

Story: the request/response models are the API's contract; keeping them in their own file means you can read the whole contract without scrolling past route code. **If not:** the Build Task's `schemas.py` would be a new idea instead of a file layout you've already used.

```
# practice/chat_schemas.py
from pydantic import BaseModel
```

```
class ChatRequest(BaseModel):
    message: str
```

```
class ChatResponse(BaseModel):
    reply: str
```

```
# practice/chat_route_practice.py
from fastapi import FastAPI, HTTPException
from chat_schemas import ChatRequest, ChatResponse

app = FastAPI()

class ChatInputError(Exception):
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
```

```

        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")

```

Expected output: identical to Approach 1 — this is purely a file-organization change.

Approach 3 — logging the real error, leaking nothing to the client

Story: Approach 1's 500 is safe for the client, but it throws the real reason away — nobody on the server ever learns what broke. `logger.exception()` keeps the full traceback in the server log, while the client still sees only a plain message. **If not:** the Build Task's "fake internal error → full trace in logs only" test case would have nothing to show in the logs.

Copy Doc01's `logging_setup.py` into `practice/` first, unchanged.

```

# practice/chat_route_practice.py
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel

from logging_setup import get_logger    # Doc01 Build Task

logger = get_logger(__name__)
app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    # why: a fake crash you can trigger on command, to test the 500 path
    if message == "boom":
        raise RuntimeError("simulated internal failure")
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)

```

```
def chat(request: ChatRequest) -> ChatResponse:
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        # how: logs the message AND the full traceback – server side only
        logger.exception("chat route failed")
        # why: the client gets a plain message, never str(exc)
        raise HTTPException(status_code=500, detail="Something went wrong.")

if __name__ == "__main__":
    client = TestClient(app)
    response = client.post("/chat", json={"message": "boom"})
    print(response.status_code, response.json())
```

Expected output (`python chat_route_practice.py`):

```
2026-01-01 00:00:00,000 __main__ ERROR chat route failed
Traceback (most recent call last):
...
RuntimeError: simulated internal failure
500 {'detail': 'Something went wrong.'}
```

The first four lines are the server log. The last line is all the client ever gets — it never sees the word "RuntimeError," the string "boom", or a stack trace. That gap is the entire point of the "no raw error trace" rule.

Approach 4 — a request ID on every log line

Story: once many requests run at the same time, their log lines mix together, and "which ERROR line belongs to the failed request?" has no answer. A short ID made once per request, and put on every log line for it, ties them together. **If not:** the Build Task's "tagged per-request" logging requirement would be the first time you ever wrote a middleware.

```
# practice/chat_route_practice.py
import uuid
from fastapi import FastAPI, HTTPException, Request
from fastapi.testclient import TestClient
from pydantic import BaseModel

from logging_setup import get_logger # Doc01 Build Task

logger = get_logger(__name__)
```

```

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    if message == "boom":
        raise RuntimeError("simulated internal failure")
    return "You said: " + message

# why: a middleware runs around EVERY request, so the ID is made
# in one place instead of inside each route
@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    # how: request.state is a spot to carry values into the route
    request.state.request_id = request_id
    response = await call_next(request) # runs the actual route
    # why: the client can quote this ID when reporting a problem
    response.headers["X-Request-ID"] = request_id
    return response

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest, http_request: Request) -> ChatResponse:
    request_id = http_request.state.request_id
    try:
        reply_text = run_chat(request.message)
        # how: the ID goes at the start of every log line
        logger.info("[%s] chat ok", request_id)
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        logger.exception("[%s] chat route failed", request_id)
        raise HTTPException(status_code=500, detail="Something went wrong.")

if __name__ == "__main__":
    client = TestClient(app)

```

```
response = client.post("/chat", json={"message": "hi"})
print(response.status_code, response.headers["X-Request-ID"])
```

Expected output (`python chat_route_practice.py` ; the ID is random each run):

```
2026-01-01 00:00:00,000 __main__ INFO [a1b2c3d4] chat ok
200 a1b2c3d4
```

The client sees the same 400/500/200 replies as Approach 3, plus an `X-Request-ID` header. Searching the logs for `a1b2c3d4` shows exactly that one request's story.

Difference from Basic: Approach 1 adds a named `ChatInputError`, so a deliberate rejection (empty message) returns 400 instead of being lumped in with real failures at 500. Approach 2 moves the models into their own file, the same split as the Build Task's `schemas.py`. Approach 3 logs the real exception in full, on the server only. Approach 4 builds on Approach 3 by tagging every log line with a per-request ID — exactly what the Build Task's "tagged per-request" logging asks for.

Which one should you actually write? Approach 1's 400/500 split is the minimum any real API needs — write that first. Add Approach 3's `logger.exception()` the moment your route can fail in a way you didn't expect (which is always, eventually) — a 500 with no server-side record is worse than a stack trace nobody sees. Add Approach 4's request ID once more than one request can run at a time — which is true for the Build Task. Approach 2's file split is worth doing whenever the models grow past a few lines.